

Problem 1: Runtime Analysis

a)

```
void f1(int n)
{
    int i=2;
    while(i < n){
        /* do something that takes O(1) time */
        i = i*i;
    }
}
```

The while loop stops when $i \geq n$, and $i=2$ at the start of the loop. Inside the loop, $i=i*i$. Assuming iteration k , $2^{2^k} \geq n$. Solve for k , $k \geq \log_2(\log_2 n)$.
 $T(n) = \Theta(\log \log n)$.

b)

```
void f2(int n)
{
    for(int i=1; i <= n; i++){
        if( (i % (int)sqrt(n)) == 0){
            for(int k=0; k < pow(i,3); k++) {
                /* do something that takes O(1) time */
            }
        }
    }
}
```

$$T(n) = \sum_{i=1}^n \Theta(1) + \sum_{\text{if true}} \Theta(i^3)$$

"If" statement is true when the remainder of $i / \sqrt{n}$ is 0 or when i is a multiple of $\sqrt{n}$. $i = k\sqrt{n}$, and there are $\sqrt{n}$ such k values.

$$T(n) = \sum_{i=1}^n \Theta(1) + \sum_{k=1}^{\sqrt{n}} \Theta((k\sqrt{n})^3)$$

$$T(n) = \Theta(n) + (\sqrt{n})^3 \sum_{k=1}^{\sqrt{n}} \Theta(k^3)$$

Using the general form of the arithmetic series,

$$T(n) = \Theta(n) + (\sqrt{n})^3 \Theta(\sqrt{n}^4)$$

$$T(n) = \Theta(n) + (\sqrt{n})^3 \Theta(n^2)$$

$$T(n) = \Theta(n^{7/2})$$

c)

```
for(int i=1; i <= n; i++){
    for(int k=1; k <= n; k++){
        if( A[k] == i){
            for(int m=1; m <= n; m=m+m){
                // do something that takes O(1) time
                // Assume the contents of the A[] array are not changed
            }
        }
    }
}
```

Looking at the most inner loop, $m = m + m$ or $m = 2*m$. The loop stops when $m > n$, so during iteration k , $m = 2^{(k-1)}$. Solve for k in $2^{(k-1)} > n$ and get $k > \log_2 n + 1$, so the runtime is $\Theta(\log n)$.

$$T(n) = \sum_{i=1}^n \sum_{k=1}^n \Theta(1) + \sum_{\text{if true}} \Theta(\log n)$$

"If" statement is true when $A[k] == i$. For all i from 1 to n , $A[k] == i$ is true at most once per k , meaning it is true at most n times because k goes up to n .

$$T(n) = \sum_{i=1}^n \sum_{k=1}^n \Theta(1) + n \Theta(\log n)$$

$$T(n) = \sum_{i=1}^n \Theta(n) + \Theta(n \log n)$$

$$T(n) = \Theta(n^2) + \Theta(n \log n)$$

$$T(n) = \Theta(n^2)$$

d)

```
int f (int n)
{
    int *a = new int [10];
    int size = 10;
    for (int i = 0; i < n; i ++)
```

```

    {
        if (i == size)
        {
            int newsize = 3*size/2;
            int *b = new int [newsize];
            for (int j = 0; j < size; j ++) b[j] = a[j];
            delete [] a;
            a = b;
            size = newsize;
        }
        a[i] = i*i;
    }
}

```

$$T(n) = \sum_{i=0}^{n-1} \Theta(1) + \sum_{\text{if true}} \Theta(\text{size at that point})$$

“If” statement evaluates to true when $i == \text{size}$. Size is updated when $i == \text{size}$ and resizing happens. Resizing happens roughly $3/2$ times the previous one due to $\text{newsize} = 3*\text{size}/2$, so it's a geometric series with a ratio of $3/2$.

$$T(n) = \sum_{i=0}^{n-1} \Theta(1) + \sum_{j=0}^m (3/2)^j$$

$$T(n) = \Theta(n) + \sum_{j=0}^m (3/2)^j$$

Using the geometric series summation formula,

$$T(n) = \Theta(n) + \Theta(3/2^m)$$

Since the final resize occurs when $\text{size} \approx n$,

$$T(n) = \Theta(n) + \Theta(n)$$

$$T(n) = \Theta(n)$$

Problem 2: Linked List Recursion Tracing

```
struct Node {
    int val;
    Node* next;
};

Node* llrec(Node* in1, Node* in2)
{
    if(in1 == nullptr) {
        return in2;
    }
    else if(in2 == nullptr) {
        return in1;
    }
    else {
        in1->next = llrec(in2, in1->next);
        return in1;
    }
}
```

Question a: What linked list is returned if llrec is called with the input linked lists in1 = 1,2,3,4 and in2 = 5,6?

iteration	in1	in2	action
1	1->2->3->4	5->6	1->next = llrec(5->6, 2->3->4)
2	5->6	2->3->4	5->next = llrec(2->3->4, 6)
3	2->3->4	6	2->next = llrec(6, 3->4)
4	6	3->4	6->next = llrec(3->4, nullptr)
5	3->4	nullptr	return 3->4

return: 1->5->2->6->3->4

Question b: What linked list is return if llrec is called with the input linked lists in1 = nullptr and in2 = 2?

iteration	in1	in2	action
1	nullptr	2	return 2

return: 2